CHAOSLAB

CHAOS ENGINEERING & RESILIENCE TESTING

An Interactive Chaos Engineering Platform for Testing Fault Tolerance, Recovery, and System Reliability.

BREAK SAFELY. OBSERVE PRECISELY. RECOVER RELIABLY.

TECHNICAL PROJECT REPORT | JUDGE VERIFICATION EDITION

Version 1.0 • 27 October 2026 • Reference implementation blueprint

REPORT STATUS

No source code, screenshots, runtime logs, or test artifacts were attached to this conversation. This document is therefore a complete, judge-ready technical blueprint and verification template, not proof that a particular application already implements the described features. All stack choices, routes, thresholds, observations, and results must be confirmed against the actual application before being presented as implemented.

Prepared to support a live demonstration, reproducible testing, and independent evaluation. Replace every “To be measured” field with evidence captured from the target environment.

02 / PROJECT ABSTRACT

CHAOSLAB is a proposed web-based control surface for running bounded, authorized failure-injection experiments against a designated test application. It gives judges a single place to select a scenario, review its impact, confirm execution, observe system behavior, verify recovery, and inspect a durable result record.

The report defines four experiments: downstream dependency outage; isolated test-database table failure; malformed result-record handling; and a bounded load test with exactly 100 total requests. It also defines a reference architecture, safe execution lifecycle, evidence schema, pass/fail rules, observability requirements, and API contract examples.

The governing principle is evidence over appearance: a red indicator or triggered exception does not establish resilience. A scenario passes only when its hypothesis, expected behavior, safety boundaries, and recovery checks are supported by timestamped artifacts from the actual run.

Status convention

Implemented and tested	Repository/runtime behavior exists and a reproducible run has supporting evidence.
Implemented, not yet tested	Code path exists, but a qualifying test run and evidence are absent.
Proposed	Design recommendation or example; not confirmed in the current application.
To be measured	Runtime value must be collected from the target environment.

This report uses the proposed project name CHAOSLAB because no application files were supplied. Preserve an existing application name and map this blueprint to its actual stack when source code becomes available.

03 / CONTENTS

Section	Topic
01	Cover page
02	Project abstract
03	Contents
04	Problem statement and motivation
05	Objectives and scope
06	Features and capabilities
07	System architecture
08	Technology stack
09	Chaos Control Center interface
10	Failure injection and recovery workflow
11	Scenario 1 — Dependency outage
12	Scenario 2 — Isolated database table failure
13	Scenario 3 — Invalid result record
14	Scenario 4 — Bounded 100-request load test
15	Observability, logging, and metrics
16	Safety controls and test isolation
17	Test execution and verification methodology
18	Results and evidence
19	Limitations and future improvements
20	Conclusion
21	Appendix: configuration, APIs, commands, checklist, glossary

Reading guide: sections 11–14 share the same scenario template. Appendix tables are intended to be filled with implementation-specific details and artifacts after a controlled run.

04 / PROBLEM STATEMENT & MOTIVATION

Distributed web applications fail in partial and often ambiguous ways: a downstream service times out, a database operation fails, malformed data reaches a read path, or a burst of requests consumes constrained resources. A system that works during normal operation may still crash, amplify an outage through retries, leak implementation details, or fail to recover after the fault is removed.

Chaos engineering makes these assumptions testable by introducing a controlled fault, observing a real application response, and comparing the outcome with an explicit hypothesis. The experiment must be limited in scope and reversible, with a known stop condition and a recovery check.

Project problem

Teams and judges often see a failure animation without enough proof that the underlying resilience mechanism was exercised. CHAOSLAB is intended to connect the control action to observable effects: request IDs, retry counters, breaker transitions, safe fallback signals, database isolation evidence, latency distributions, and post-experiment health.

Engineering principles

• Use an explicit hypothesis and measurable pass/fail criteria before each run.

• Constrain blast radius to a local or dedicated test environment.

• Prefer reversible fault toggles and disposable fixtures over destructive operations.

• Bound retries, concurrency, timeouts, total request count, and experiment duration.

• Separate expected experiment failure from platform failure and record both.

• Never label an outcome “recovered” until health checks and functional probes pass.

05 / OBJECTIVES & SCOPE

Objectives

• Provide an authorized Chaos Control Center for selecting and monitoring experiments.

• Demonstrate retry, circuit breaker, and fallback behavior during dependency unavailability.

• Prove database fault tests operate only on an isolated disposable database/schema.

• Validate rejection or quarantine of malformed records without exposing sensitive internals.

• Run exactly 100 requests against a designated test endpoint under bounded concurrency and timeout.

• Persist scenario configuration, execution timestamps, logs, metrics, recovery checks, and verdicts.

• Make the judge's verification steps reproducible without trusting a UI status alone.

In scope

Control-plane design, scenario contracts, fault isolation, observability, evidence collection, and a judge verification procedure. The report includes proposed endpoints and sample commands; they are not asserted to exist until confirmed in source code.

Out of scope

Unrestricted traffic generation, production fault injection by default, destructive database experiments on shared data, claims of production-scale capacity based on 100 requests, and fabricated benchmarks or screenshots.

Acceptance gate

A scenario is eligible for a live demo only after authorization, environment identity, bounded parameters, stop conditions, and a recovery plan have been verified. If the application cannot enforce those controls, the scenario remains proposed and should not be executed against a shared or production system.

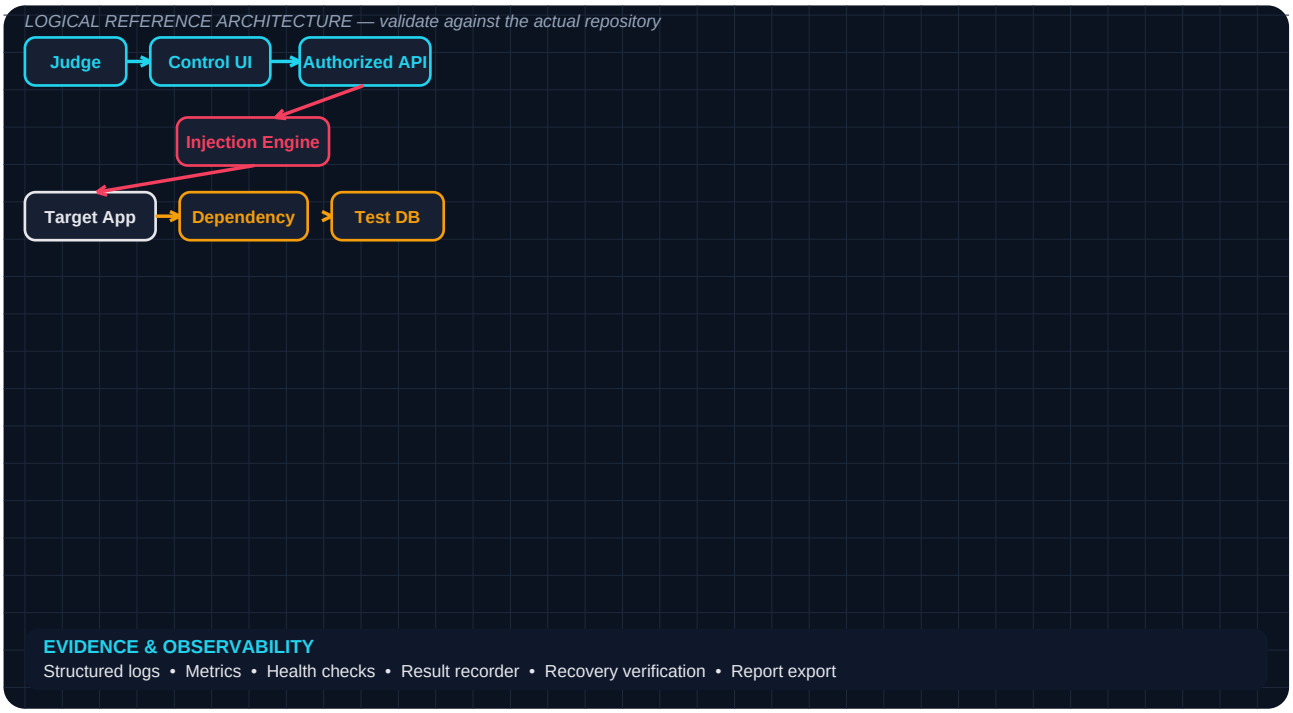
06 / FEATURES & CAPABILITIES

Capability	Expected behavior	Verification signal
Chaos controls	Four separately configured scenario actions with confirmation before execution.	Scenario ID, actor, config hash, timestamp.
Health overview	Displays target health and last successful functional probe.	Health endpoint plus independent probe result.
Dependency resilience	Shows retry attempt count, breaker state, fallback activation.	Correlated logs and dependency stub state.
Database isolation	Uses disposable DB/schema and denies production targets.	Environment guard and connection identity evidence.
Invalid-record handling	Validates required fields/types; rejects or quarantines invalid data.	Validation event and before/after counts.
Bounded load test	Exactly 100 total request attempts; bounded concurrency and timeouts.	Issued-request count equals 100.
Event log	Append-only, timestamped events correlated by run ID.	Exportable structured records.
Recovery verification	Runs health and functional checks after fault removal.	Recovery result with timestamp and probe details.
Report export	Produces a result summary and evidence references.	Machine-readable JSON/CSV plus readable report.

Implementation status: all rows above describe required or proposed behavior. The repository was not supplied, so current implementation status is not verified.

07 / SYSTEM ARCHITECTURE

The diagram is a logical reference architecture. It is intentionally technology-neutral and must be reconciled with the actual repository, deployment topology, and data stores.



Logical components

Component	Responsibility	Boundary / note
Judge / operator	Selects scenario, reviews impact, confirms execution.	Authenticated role; no direct database access.
Control panel	Displays health, parameters, status, events, and evidence.	UI is not the security boundary.
Authorized Chaos API	Validates identity, role, environment, parameters, and limits.	Server-side authorization on every action.
Failure injection engine	Applies reversible faults and enforces duration/stop limits.	Allow-listed actions only; idempotent cleanup.
Target application	Runs the actual request path under test.	Separate from the control plane where feasible.
Dependency / test DB	Test double or isolated disposable data store.	No production credentials or shared schema.
Evidence pipeline	Correlated logs, metrics, health probes, result recorder.	Timestamped, redacted, exportable artifacts.

Request lifecycle

The operator requests a run. The API authenticates the actor, verifies the environment and scenario policy, validates bounded parameters, writes an audit event, and returns a run ID. The engine injects the selected fault. The application handles requests through normal resilience code. Monitoring captures correlated events and measurements. The engine removes the fault, waits for a bounded stabilization window, executes health and functional probes, then records a verdict and evidence references.

08 / TECHNOLOGY STACK

No source code or dependency manifest was supplied. The stack below is therefore a selection checklist, not a claim about an existing implementation.

Layer	Candidate options	Confirm from repository
Frontend	React, Vue, Svelte, server-rendered templates	Framework, router, UI component library
Backend/API	Node.js/Express, Python/FastAPI, Java/Spring, .NET	Runtime, route definitions, auth middleware
Persistence	PostgreSQL, MySQL, SQLite, document store	Driver, schema, test DB strategy
Resilience	Retry policy, circuit breaker library, fallback adapter	Actual retry limits, breaker thresholds, reset behavior
Observability	Structured JSON logs, Prometheus/OpenTelemetry, health probes	Instrumentation and export paths
Testing	Unit/integration tests, HTTP client, load harness	Test commands, fixtures, coverage
Deployment	Local containers, CI, test namespace	Environment isolation and deployment guardrails

Stack discovery checklist

• Inspect package/dependency manifests and lockfiles.

• Locate route registrations, authentication/authorization middleware, and health endpoints.

• Trace the dependency client and determine whether retry and breaker behavior is actually configured.

• Identify database connection configuration without copying secret values into the report.

• Locate tests, CI workflows, container definitions, environment guards, and logging configuration.

• Record exact versions and reproducible commands in the implementation-specific revision.

Never publish secrets, connection strings, access tokens, private environment variables, or sensitive record contents. Document variable names only when useful, with values redacted.

09 / CHAOS CONTROL CENTER

The dashboard should make system state, experiment state, and evidence state distinct. A control being available does not mean it is safe to run; a fault being triggered does not mean resilience passed.

CHAOS CONTROL CENTER Environment: TEST-ONLY • Run ID: run_xxx	**HEALTH OVERVIEW** App: UNKNOWN Dependency: UNKNOWN Test DB: UNKNOWN
DEPENDENCY OUTAGE Retry: 0 / configured max Breaker: CLOSED / OPEN / HALF-OPEN Fallback: inactive / active	**REQUEST METRICS** Issued: 0 / 100 Success: To be measured Failure: To be measured p95 latency: To be measured
DATABASE SAFETY Target DB identity verified: No / Yes Production access denied: No / Yes	**RECOVERY VERIFICATION** Fault removed: Pending Health probe: Pending Functional probe: Pending Verdict: NOT RUN
LIVE EVENT LOG [timestamp] INFO run accepted [timestamp] WARN retry scheduled [timestamp] ERROR injected fault observed Illustrative event shapes only; not real logs.	**CONTROLS** [Confirm] Run scenario [Cancel] Stop if supported [Verify recovery] Probe target

Required interface behavior

• Each scenario card shows purpose, target, expected impact, preconditions, timeout, and cleanup plan before confirmation.

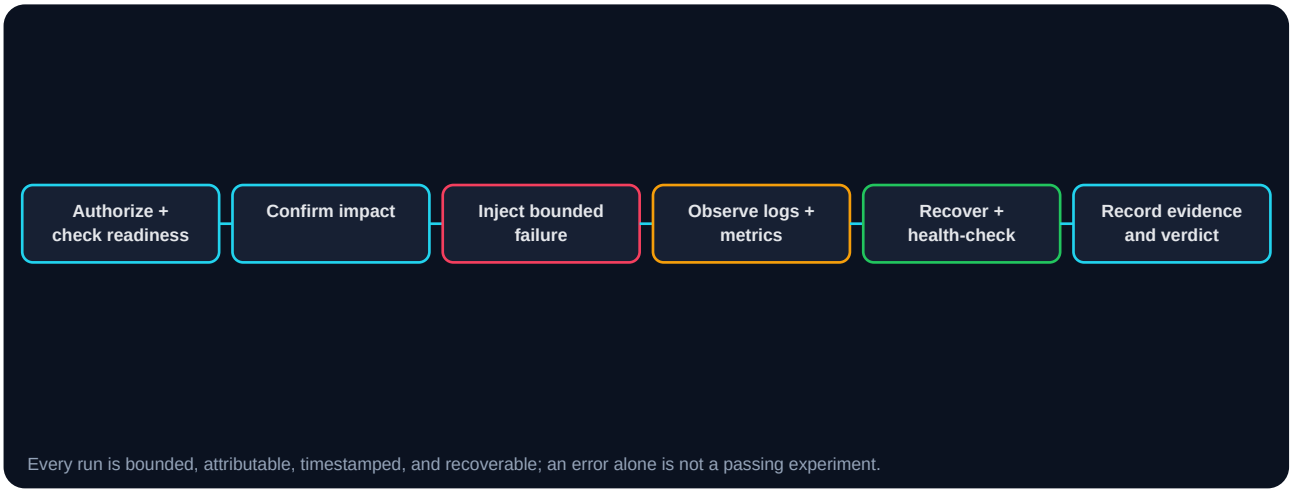
• Execution states: NOT RUN, VALIDATING, RUNNING, STOPPING, RECOVERING, PASSED, FAILED, ABORTED.

• Show labels and text in addition to color: green means verified healthy, amber means degraded/retrying, red means fault/critical, cyan means active/observing.

• Disable conflicting experiments when a run is active; prevent double submission and use an idempotency key.

• Expose a run detail view with configuration, timeline, evidence links, and the reason for the verdict.

10 / FAILURE INJECTION & RECOVERY WORKFLOW



Every run is bounded, attributable, timestamped, and recoverable; an error alone is not a passing experiment.

Step	Action	Required evidence / guard
1	Judge opens panel	Authenticated session and role.
2	Check readiness	Environment label, target allow-list, DB isolation, no conflicting run.
3	Select scenario	Scenario ID and parameter validation.
4	Confirm impact	Human confirmation with expected blast radius and stop conditions.
5	Inject bounded fault	Run ID, start time, injected fault, expiry/cleanup deadline.
6	Observe behavior	Structured logs, counters, request results, health probes.
7	Attempt recovery	Fault removal and resilience behavior recorded.
8	Verify recovery	Health and functional probes pass within configured window.
9	Record result	Immutable/append-only audit event and evidence manifest.
10	Judge reviews	Pass/fail rule, metrics, artifacts, and limitations.

Emergency stop semantics

Where technically supported, cancel should stop new work, signal in-flight workers, restore toggles or dependency stubs, close load generators, and enter RECOVERING rather than claiming immediate success. Cancellation itself must be logged. A hard process kill is not a substitute for cleanup. If an action cannot be safely cancelled, the UI must disclose this before execution.

11 / SCENARIO 1 — DEPENDENCY OUTAGE

Scenario ID: CH-01 • Dependency outage

Objective: Verify that downstream unavailability is handled without uncontrolled retry amplification, and that fallback behavior and recovery are observable.

Failure hypothesis: When the downstream dependency fails, the application should retry only retryable failures within a configured budget, open the circuit after its threshold, return a clearly identified safe fallback where valid, and recover after the dependency is restored.

Preconditions

- Use a test dependency, stub, or fault proxy; never disable a shared production service.
- Record retry limit, backoff, jitter, breaker threshold/window, open duration, and fallback policy.
- Ensure the target endpoint and test identity are allow-listed.

Injection method

- Toggle the test dependency to return a controlled timeout or retryable 5xx.
- Keep fault duration short and bounded; preserve an automatic cleanup deadline.

Expected application behavior

- Requests complete within configured timeouts; fallback is explicit and does not misrepresent stale data as fresh.
- Breaker transitions CLOSED → OPEN and later HALF-OPEN → CLOSED only when probes satisfy policy.
- Non-retryable errors are not retried.

Resilience mechanism under test

- Bounded retry policy; exponential backoff with jitter if implemented; circuit breaker; safe fallback; health probe.

Execution steps

1. Capture baseline health and a successful dependency request.
2. Enable the controlled dependency fault and execute a small, bounded probe.
3. Inspect retry attempts, classified errors, breaker transitions, and fallback marker.
4. Restore dependency; wait for breaker recovery policy; issue a functional probe.
5. Export logs, metrics, and run record.

Expected logs and metrics

- run_id and correlation/request ID; retry attempt/max; retryable classification; breaker state transitions; fallback_used; latency and error counters.

Pass/fail criteria

- PASS only if attempts do not exceed configured maximum, breaker behavior matches configured policy, fallback is correctly identified, and post-restore health plus functional probe pass.
- FAIL if retries exceed the budget, fallback is misleading/unsafe, the application crashes, or recovery does not meet its declared window.

Recovery procedure

- Disable fault toggle; verify dependency responds; allow breaker policy to progress; run health and functional probes; confirm no residual fault.

Evidence required

- Before/during/after timestamps; effective configuration; correlated logs; breaker/fallback metrics; dependency stub state; recovery probe result.

Risks and safety controls

- Retry storms, cascading load, stale data confusion. Use a test dependency, short duration, jitter, strict retry cap, and automatic fault expiry.

Evidence record — populate from an actual run

Test ID	Timestamp	Configuration	Triggered failure
CH-01	To be measured	To be measured	To be recorded
Expected behavior	Observed behavior	Logs / screenshot ref	Metrics
As defined above	To be measured	To be attached	To be measured
Recovery status	Final verdict	Operator / run ID	Environment
To be measured	NOT RUN	To be recorded	TEST ONLY

12 / SCENARIO 2 — ISOLATED DATABASE TABLE FAILURE

Scenario ID: DB-02 • Isolated test database table failure

Objective: Verify controlled handling of a missing or inaccessible table without damaging production data or unrelated database operations.

Failure hypothesis: If a test-only table becomes unavailable, the affected request returns a controlled error, the service remains alive, sensitive SQL/connection details are not exposed, and restoring the table returns the path to normal.

Preconditions

- Use a disposable database/container or dedicated schema and test-only credentials.
- At startup, assert the database identity matches an explicit test allow-list; fail closed otherwise.
- Seed known fixtures and record baseline row counts.

Injection method

- Prefer a test-only schema migration/fixture that omits or renames a dedicated test table, or a DB proxy that denies access to it.
- Do not drop or rename production tables; do not reuse production credentials.

Expected application behavior

- Affected operation returns a documented controlled error response and safe correlation ID.
- Application process and unrelated health checks remain available where architecture permits.
- Logs contain internal diagnostic context but public response does not expose raw SQL or credentials.

Resilience mechanism under test

- Error boundary, transaction handling, connection timeout policy, readiness/liveness separation, isolation controls.

Execution steps

1. Verify target DB identity and capture baseline counts.
2. Apply the isolated table fault through a disposable fixture or test-only proxy.
3. Call the affected endpoint and an unrelated safe endpoint.
4. Capture status code, sanitized response, server logs, and database identity.
5. Restore schema/fixture; rerun both probes and compare expected counts.

Expected logs and metrics

- db_fault_injected; operation name; sanitized error class; request ID; HTTP status; connection health; baseline/final counts.

Pass/fail criteria

- PASS only if the intended path fails in a controlled way, no production target is reachable, unrelated safe operations behave as expected, and restored schema passes probes.
- FAIL if environment identity cannot be proved, production access is possible, process crashes unexpectedly, or raw internals leak.

Recovery procedure

- Recreate/restore the test schema from migrations or disposable snapshot; verify row counts and health; destroy the test environment if needed.

Evidence required

- DB target identity evidence; migration/fixture identifier; baseline and final counts; HTTP response; sanitized logs; restoration probe; screenshot/run export.

Risks and safety controls

- Wrong-target destructive action, shared schema impact, data loss. Enforce test-only credentials, database allow-list, disposable instance, explicit confirmation, and cleanup verification.

Evidence record — populate from an actual run

Test ID	Timestamp	Configuration	Triggered failure
DB-02	To be measured	To be measured	To be recorded
Expected behavior	Observed behavior	Logs / screenshot ref	Metrics
As defined above	To be measured	To be attached	To be measured
Recovery status	Final verdict	Operator / run ID	Environment
To be measured	NOT RUN	To be recorded	TEST ONLY

13 / SCENARIO 3 — INVALID RESULT RECORD

Scenario ID: DATA-03 • Invalid result record

Objective: Prove that malformed data is detected and rejected or quarantined safely, while valid records remain accessible.

Failure hypothesis: A record with a missing required field, invalid type, or out-of-range value is rejected at the validation boundary or moved to a quarantine path; the application does not crash or expose sensitive internals.

Preconditions

- Use isolated test storage and synthetic, non-sensitive fixture data.
- Define the schema, required fields, type constraints, and expected rejection/quarantine policy.
- Capture record counts before insertion.

Injection method

- Insert one or more deliberately invalid records through a test fixture or controlled test-only endpoint.
- Cover at least one required-field violation and one type violation if supported.

Expected application behavior

- Invalid record is rejected or quarantined according to design; no malformed record is treated as valid output.
- Valid records remain queryable; user-facing errors are safe and useful.
- Counts and validation events make the handling auditable.

Resilience mechanism under test

- Schema validation, type checking, constraints, safe serialization, quarantine/dead-letter handling where implemented.

Execution steps

1. Record baseline total/valid/quarantined counts.
2. Submit the malformed fixture through the normal ingestion path or documented test hook.
3. Query the validation result and ensure the invalid record is not returned as valid data.
4. Query a known valid fixture and confirm accessibility.
5. Record after-counts and remove test fixtures.

Expected logs and metrics

- validation_failed; field/path; rule identifier; record correlation ID (not sensitive payload); accepted/rejected/quarantined counts.

Pass/fail criteria

- PASS only if invalid data is rejected/quarantined as specified, valid fixture remains accessible, counts reconcile, and no sensitive details leak.
- FAIL if malformed data is silently accepted as valid, valid data is corrupted, or the application crashes.

Recovery procedure

- Delete synthetic fixtures or reset disposable store; rerun valid-record query; verify count reconciliation.

Evidence required

- Schema version; sanitized fixture; before/after counts; validation log; endpoint result; valid-record query result; recovery check.

Risks and safety controls

- Accidental real-data insertion, sensitive payload logging, false rejection of valid records. Use synthetic fixtures, redaction, scoped test identity, and cleanup.

Evidence record — populate from an actual run

Test ID	Timestamp	Configuration	Triggered failure
DATA-03	To be measured	To be measured	To be recorded
Expected behavior	Observed behavior	Logs / screenshot ref	Metrics
As defined above	To be measured	To be attached	To be measured
Recovery status	Final verdict	Operator / run ID	Environment
To be measured	NOT RUN	To be recorded	TEST ONLY

14 / SCENARIO 4 — BOUNDED 100-REQUEST LOAD TEST

The load test must issue exactly 100 total request attempts to an explicitly designated test endpoint. Retries by the load generator are disabled unless they are counted inside the same 100-attempt budget. This is a bounded smoke/load experiment, not a production-capacity certification.

Scenario ID: LOAD-04 • Bounded 100-request test

Objective: Measure basic response behavior under a small, controlled request burst and confirm the application remains healthy afterward.

Failure hypothesis: Under a fixed total of 100 request attempts and bounded concurrency/timeouts, the test endpoint completes or fails each request within the declared budget, metrics reconcile to 100 attempts, and health checks remain acceptable or recover within the declared window.

Preconditions

- Use an owned test endpoint and test environment; confirm endpoint is safe and idempotent.
- Configure concurrency, request timeout, global duration limit, and cancellation behavior.
- Ensure the harness has a hard total-attempt cap of 100.

Injection method

- Generate exactly 100 request attempts to the target endpoint; enforce the cap in the harness and server-side policy where possible.
- Use bounded concurrency (recommended initial setting: 5, subject to environment review) and a per-request timeout (example: 3 seconds; configure for the actual endpoint).
- Disable automatic client retries or count every retry as one of the 100 attempts.

Expected application behavior

- All attempts are accounted for as success or failure; no more than 100 requests are issued.
- Latency percentiles and throughput are computed from captured timings; p99 is reported cautiously with only 100 samples.
- Health and functional probes are recorded before and after the run.

Resilience mechanism under test

- Load harness; request timeout; concurrency semaphore; metrics collector; health probes; cancellation token.

Execution steps

1. Capture baseline health and resource usage if available.
2. Confirm endpoint, concurrency and timeout configuration; preview impact.
3. Run the fixed 100-attempt test and retain per-request timestamps/status/latency.
4. Calculate success/failure totals, average, p50/p95/p99 where tooling supports them, throughput and error rate.
5. Run post-test health and functional probes; export the result.

Expected logs and metrics

- issued_total exactly 100; success_total + failure_total = 100; per-request latency; status distribution; p50/p95/p99; throughput; error rate; resource snapshots if available.

Pass/fail criteria

- PASS only if exactly 100 attempts are accounted for, the test remains within configured concurrency/timeouts, no safety limit is violated, and post-test health checks pass.
- Performance SLO verdict is separate and must use a predeclared threshold; do not invent one after seeing results.

Recovery procedure

- Stop new scheduling at 100 attempts; cancel on global deadline; wait for in-flight requests to settle or time out; verify health and functional probes; capture final resources.

Evidence required

- Harness command/config; request-level result file; summary metrics; timestamps; app logs; pre/post health; cancellation/limit evidence.

Risks and safety controls

- Resource saturation or accidental external targeting. Use allow-listed local/test endpoint, concurrency cap, timeout, global deadline, no unrestricted loops, and cancel control.

Evidence record — populate from an actual run

Test ID	Timestamp	Configuration	Triggered failure
LOAD-04	To be measured	To be measured	To be recorded
Expected behavior	Observed behavior	Logs / screenshot ref	Metrics
As defined above	To be measured	To be attached	To be measured
Recovery status	Final verdict	Operator / run ID	Environment
To be measured	NOT RUN	To be recorded	TEST ONLY

15 / OBSERVABILITY, LOGGING & METRICS

Observability must connect the control action to the application outcome. Every experiment should carry a unique run_id and, where available, a request/correlation ID across the control API, target application, dependency adapter, and result recorder.

Structured log event shape (illustrative only)

```
{
  "timestamp": "2026-01-01T12:00:00Z",
  "level": "WARN",
  "event": "retry_scheduled",
  "run_id": "run_example",
  "request_id": "req_example",
  "scenario_id": "CH-01",
  "attempt": 2,
  "max_attempts": 3,
  "dependency": "test-dependency",
  "outcome": "retryable_timeout"
}
```

The timestamp and identifiers above are illustrative placeholders, not captured application logs.

Minimum metric set

Metric	Definition / use
chaos_run_total	Runs by scenario and terminal state.
chaos_injection_active	Gauge showing whether a fault toggle is active.
dependency_retry_attempts_total	Count of retry attempts, tagged by classified outcome.
circuit_breaker_state	Current state; log transitions with timestamps.
fallback_activations_total	Fallback responses actually returned.
http_requests_total	Request count by route/status class; avoid high-cardinality labels.
http_request_duration_seconds	Latency histogram for percentiles.
chaos_load_issued_total	Request attempts issued by the bounded harness.
chaos_validation_rejected_total	Invalid records rejected/quarantined.
recovery_probe_success	Result of post-fault health and functional probes.

Health endpoint semantics

Separate liveness (process can run) from readiness (dependencies required to serve traffic are available). During an intentional dependency fault, readiness behavior should match the product's service contract; a liveness failure should not be used as a generic signal for every downstream problem. Health endpoints should not disclose credentials, stack traces, or topology details to unauthenticated clients.

16 / SAFETY CONTROLS & TEST ISOLATION

Control	Required implementation	Judge verification
RBAC	Only authorized roles can start, cancel, or inspect sensitive run details.	Try unauthenticated and insufficient-role requests; both must be denied.
Environment guard	Explicit test environment marker plus target allow-list; fail closed on ambiguity.	Show sanitized environment identity and rejection of a non-test target.
Database isolation	Dedicated disposable instance/schema and test-only credentials.	Verify connection identity and lack of production credential access.
Execution lease	One active destructive experiment per target unless explicitly proven safe.	Attempt overlapping run and verify it is rejected or safely serialized.
Hard bounds	Maximum duration, retries, concurrency, total requests, and payload sizes.	Inspect server-side enforcement, not only UI controls.
Audit log	Actor, scenario, target alias, config, start/end, cancel, cleanup, verdict.	Export a run timeline with correlated IDs.
Cleanup	Automatic expiry, finally/cleanup path, restoration verification.	Demonstrate fault removed after normal completion and cancellation.
Secret hygiene	Redact tokens, connection strings, raw invalid payloads, and sensitive fields.	Inspect logs and exported reports.
Emergency stop	Cancel pending work and trigger safe cleanup where supported.	Demonstrate stop behavior in a test run.

Critical safety rule

Do not enable a database failure scenario unless the target database identity is programmatically verified against an allow-list and the operation is constrained to a disposable test target. A UI label saying “TEST” is not a sufficient isolation control.

17 / TEST EXECUTION & VERIFICATION METHODOLOGY

Use the same sequence for each experiment to make results comparable and reproducible.

• Prepare: identify build/version, environment, actor, target alias, configuration, and preconditions.

• Declare hypothesis: state what should happen, how it will be measured, and the pass/fail rule before execution.

• Baseline: capture health, dependency status, relevant counts, and baseline metrics.

• Authorize: verify role, target allow-list, isolation, bounds, and no conflicting experiment.

• Execute: confirm impact and start the scenario; record run ID and timestamps.

• Observe: capture logs, metrics, response status, breaker/retry/fallback signals, and resource measurements where available.

• Recover: remove the fault, restore fixtures, stop generators, and wait for stabilization within a declared limit.

• Verify: run health and functional probes; compare pre/post state and expected counts.

• Verdict: apply predeclared criteria; label inconclusive runs as INCONCLUSIVE rather than PASS.

• Package: export configuration, raw result data, logs, screenshot references, and report.

Verdict rules

Verdict	Meaning
PASS	Every required criterion is supported by evidence and safety/recovery checks pass.
FAIL	One or more required criteria failed or a safety invariant was violated.
INCONCLUSIVE	Evidence is missing, instrumentation failed, or the run cannot support a defensible verdict.
ABORTED	Operator or safety mechanism stopped the run; recovery must still be verified.
NOT RUN	No execution evidence exists.

A successful experiment demonstrates both the intended fault response and recovery. If recovery verification is missing, the run cannot be called a successful resilience experiment.

18 / RESULTS & EVIDENCE

No actual application artifacts were supplied. Consequently, this report intentionally records no successful experiments, benchmark values, screenshots, or observed recovery claims. Complete the tables below from the target application's actual runs.

Scenario	Implementation status	Test status	Evidence presently available
CH-01 Dependency outage	Not verified	NOT RUN	None supplied
DB-02 Test DB table failure	Not verified	NOT RUN	None supplied
DATA-03 Invalid record	Not verified	NOT RUN	None supplied
LOAD-04 100 requests	Not verified	NOT RUN	None supplied

100-request result sheet

Field	Value
Total requests	100 (hard cap; actual issued count must be verified)
Target endpoint	To be measured / confirm actual endpoint
Concurrency limit	To be measured
Request timeout	To be measured
Warm-up policy	To be measured / not applicable if disabled
Successful requests	To be measured
Failed requests	To be measured
Average latency	To be measured
p50 / p95 / p99 latency	To be measured; report method and sample count
Requests per second	To be measured
Error rate	To be measured
Pre/post health status	To be measured
Resource usage	To be measured where instrumentation exists
Final verdict	NOT RUN

For 100 attempts, error rate = failed attempts / 100 × 100%. Throughput should state the measurement interval and whether it includes client scheduling or only completed responses. Percentiles should be computed from per-request measurements and accompanied by sample count; p99 is a coarse estimate with 100 samples.

19 / LIMITATIONS & FUTURE IMPROVEMENTS

Current limitations

• No repository, source files, screenshots, runtime output, or architecture artifacts were available for inspection.

• Technology stack, actual route names, retry settings, breaker thresholds, and database schema are unconfirmed.

• Dashboard and log excerpts in this report are design examples, not screenshots or observed telemetry.

• No performance or recovery results can be stated until experiments run against the real application.

• A 100-request bounded test is useful for smoke testing and basic stability checks, not proof of production-scale capacity.

Recommended improvements

• Add signed or tamper-evident evidence manifests with artifact hashes and build identifiers.

• Integrate OpenTelemetry traces and metrics for cross-service request correlation.

• Add scenario policy files with versioned thresholds and explicit approval for higher-risk actions.

• Run experiments in ephemeral environments created from infrastructure-as-code.

• Add CI regression tests for retry caps, breaker transitions, schema validation, and cleanup paths.

• Support automatic evidence export (JSON + human-readable PDF) from each run.

• Introduce SLO-based abort conditions and a global experiment lease/lock.

• Add a dry-run preview that proves target and limits before injection.

Maturity progression

Level	Capability
1 — Observable	Run IDs, logs, health probes, basic results.
2 — Controlled	RBAC, target allow-list, bounded duration/concurrency, cleanup.
3 — Verifiable	Predeclared criteria, reproducible fixtures, evidence manifests.
4 — Automated	CI experiments, ephemeral environments, regression thresholds.
5 — Governed	Risk policy, approval workflows, SLO guardrails, audit retention.

20 / CONCLUSION

CHAOSLAB defines a practical pattern for demonstrating resilience with controlled, observable, and reversible experiments. Its value is not the ability to trigger failure by itself; the value is proving that the application responds within defined limits, protects data, avoids cascading behavior, and returns to a healthy state.

The four required scenarios cover dependency resilience, database fault containment, invalid-data handling, and a bounded 100-request load test. Each scenario has a hypothesis, preconditions, injection plan, expected behavior, pass/fail rules, recovery procedure, and evidence checklist.

Before a judging session, replace the proposed architecture and endpoint examples with the real implementation, run each scenario in an isolated test environment, and attach genuine screenshots, logs, configuration, and metrics. Leave any unsupported claim marked as proposed or not tested.

Engineering promise

Break safely. Bound the experiment and protect the environment.
Observe precisely. Correlate the fault with real behavior and measurements.
Recover reliably. Verify health and function after the fault is removed.

21 / APPENDIX A — CONFIGURATION TEMPLATE

Example values below are illustrative starting points only. Replace them with values validated for the actual application and environment.

```
# Illustrative policy; not a live configuration
environment: test-only
max_active_experiments: 1
max_experiment_duration_seconds: 60
dependency_outage:
  max_attempts: 3
  backoff: exponential_with_jitter   # only if implemented
  circuit_breaker:
    failure_threshold: "To be configured"
    open_duration_seconds: "To be configured"
  fallback_policy: "explicit safe response"
database_failure:
  allowed_target_aliases: ["disposable-test-db"]
  production_targets_allowed: false
invalid_record:
  fixture_source: "synthetic-test-fixture"
  policy: "reject or quarantine as implemented"
load_test:
  total_requests: 100
  concurrency_limit: 5              # example; review before use
  request_timeout_seconds: 3         # example; tune to endpoint
  client_retries: 0
  target_allowlist: ["designated-test-endpoint"]
```

Configuration safety

Enforce limits on the server, not only in the browser. Parse numeric inputs strictly, reject unknown scenario IDs, apply a global deadline, and ensure the load generator counts every issued attempt toward the 100-request cap.

APPENDIX B — PROPOSED API CONTRACTS

The following route names are proposed examples only. Inspect route registration and API documentation to determine actual endpoints before using them.

Proposed route	Purpose	Request / response notes
POST /api/chaos/dependency-outage	Start CH-01	Body: target alias, duration, policy ID, idempotency key. Response: run_id, accepted config, state.
POST /api/chaos/database-failure	Start DB-02	Body: disposable DB alias, fixture/fault mode, expiry. Must reject unapproved DB targets.
POST /api/chaos/invalid-record	Start DATA-03	Body: fixture ID and validation case. Never accept arbitrary production payloads.
POST /api/chaos/load-test	Start LOAD-04	Body: endpoint alias, total_requests=100, concurrency, timeout. Server enforces exact cap.
GET /api/chaos/status	Read active state	Return current run state, safe counters, and timestamps.
GET /api/health	Health probe	Return minimal liveness/readiness status; no sensitive details.
GET /api/chaos/results/{run_id}	Read result evidence	Return scenario, config, metrics, verdict, and artifact references subject to RBAC.

APPENDIX C — SAMPLE COMMANDS & RESPONSE SHAPES

These are conceptual examples. They will not work unless equivalent routes and payload schemas exist in the actual application. Use a test base URL and an authorized token managed outside source control.

```
# Conceptual example only — verify actual route and schema
curl -X POST "$TEST_BASE_URL/api/chaos/load-test" \
  -H "Authorization: Bearer $TEST_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "target": "approved-test-endpoint",
    "total_requests": 100,
    "concurrency": 5,
    "timeout_seconds": 3,
    "client_retries": 0
  }'

# Illustrative response shape only
{
  "run_id": "run_example",
  "scenario_id": "LOAD-04",
  "state": "ACCEPTED",
  "total_requests": 100,
  "target": "approved-test-endpoint"
}
```

Expected API behavior

• 401/403 for unauthenticated or unauthorized requests; 409 for conflicting runs; 422 for invalid parameters; 429 where policy rate limits apply.

• Return a run_id and explicit accepted configuration; never silently clamp an invalid request without disclosing the effective value.

• Use structured errors with a safe code, human-readable message, and correlation ID. Keep stack traces and raw SQL in protected server logs only.

• Support idempotency for start requests to prevent duplicate experiments from repeated clicks.

• Use authenticated, role-protected result retrieval and redact secrets from exports.

APPENDIX D — JUDGE EVIDENCE CHECKLIST

• Repository/build identity and actual stack/route map.

• Environment identity and proof that the target is test-only.

• Role/authorization check and audit record.

• Scenario configuration, thresholds, timeout, concurrency, and target alias.

• Pre-run health and baseline measurements.

• Run ID, start/end timestamps, actor, and correlation IDs.

• Raw structured logs and per-request metrics, not just a dashboard screenshot.

• Dependency outage: retry counts, breaker transitions, fallback marker, restoration probe.

• Database failure: isolated DB identity, baseline/final counts, sanitized error, schema restoration.

• Invalid record: schema/rule ID, rejected/quarantined count, valid record remains accessible.

• Load test: exactly 100 issued attempts, success + failure = 100, latency samples, throughput window, error rate, pre/post health.

• Recovery verification timestamp and functional probe result.

• Final verdict with predeclared criteria; inconclusive evidence is not a pass.

• Redaction review: no tokens, credentials, connection strings, or sensitive payloads.

- Exported evidence manifest with stable artifact references.

APPENDIX E — GLOSSARY

Term	Definition
Chaos engineering	Disciplined experimentation that tests system behavior under controlled failure conditions.
Fault injection	Deliberately introducing a bounded fault to exercise a real handling path.
Retry	A repeated attempt after a classified transient failure; must be bounded to avoid amplification.
Exponential backoff	A retry delay that increases between attempts; jitter randomizes timing to reduce synchronized retry bursts.
Circuit breaker	A state machine that temporarily blocks calls after a failure threshold; commonly CLOSED, OPEN, and HALF-OPEN.
Fallback	An alternate response or behavior used when the primary path is unavailable; must be explicit about freshness and completeness.
Latency	Elapsed time between a defined request start and completion event.
Throughput	Completed work per unit time over a stated measurement interval.
Error rate	Failed attempts divided by total attempts, expressed as a percentage.
Recovery time	Time from fault removal or recovery trigger to verified restoration, with the start/end definition stated.
Liveness	Whether a process is running and able to make progress.
Readiness	Whether a service is currently able to serve its intended traffic.
Blast radius	The scope of components, data, and users potentially affected by an experiment.
Evidence manifest	An index of run configuration and artifacts, ideally including hashes and provenance.

FINAL RELEASE GATE

Before presenting this report as a record of an implemented project, reconcile every proposed element with the source repository, update actual endpoint names and configuration, attach genuine run artifacts, and replace “To be measured” fields. Until then, this document is a professional reference design and verification plan—not a claim of completed tests.

CHAOSLAB • BREAK SAFELY. OBSERVE PRECISELY. RECOVER RELIABLY.